

	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

ÍNDICE

1. Arquitectura del Sistema	2
2. Tecnologías Utilizadas	2
3. Estructura del Código	2
4. Estructura de Datos y Modelo Entidad-Relación	4
5. Requisitos Funcionales y No Funcionales	8

	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

1. Arquitectura del Sistema

El **proyecto ChefManager** ha sido concebido utilizando una **arquitectura Cliente-Servidor** (BaaS). Al depender de servicios externos para la gestión de base de datos y autenticación, logramos una reducción considerable en tiempos de mantenimiento del backend.

- **Frontend (Cliente):** Aplicación de una sola página (**SPA**) desarrollada con **React 19**.
- **Backend as a Service (BaaS):** **Supabase** es la columna vertebral de persistencia de datos (PostgreSQL), así como gestor de la capa de autenticación, políticas de seguridad en base de datos (RLS) y APIs en tiempo real.

2. Tecnologías Utilizadas

- **React y Vite:** Núcleo de la aplicación. Vite optimiza de manera drástica el entorno de desarrollo y la compilación frente a otros bundlers como Webpack.
- **React Router:** Configuración de rutas estáticas y privadas.
- **Tailwind CSS:** Estilización mediante clases de utilidad atómicas orientadas a Mobile-First.
- **TypeScript:** Superset de JavaScript (se refleja el uso en la extensión de la configuración, .tsx), lo que prevé tipado fuerte, evitando errores en el tiempo de ejecución.
- **Supabase:** Integración por parte del cliente con la instancia en la nube.
- **Framer Motion:** Manejo de micro-animaciones.

3. Estructura del Código

- **Raíz del proyecto (Aplicativo/)** Contiene los archivos de configuración globales: index.html (punto de entrada HTML), package.json (dependencias y scripts), vite.config.ts (configuración del bundler), tsconfig.json con sus variantes para app y node, eslint.config.js y .gitignore.
- **src/** Código fuente completo de la aplicación. Contiene main.tsx (punto de entrada de React) y App.tsx (definición de rutas y sistema de protección de acceso).

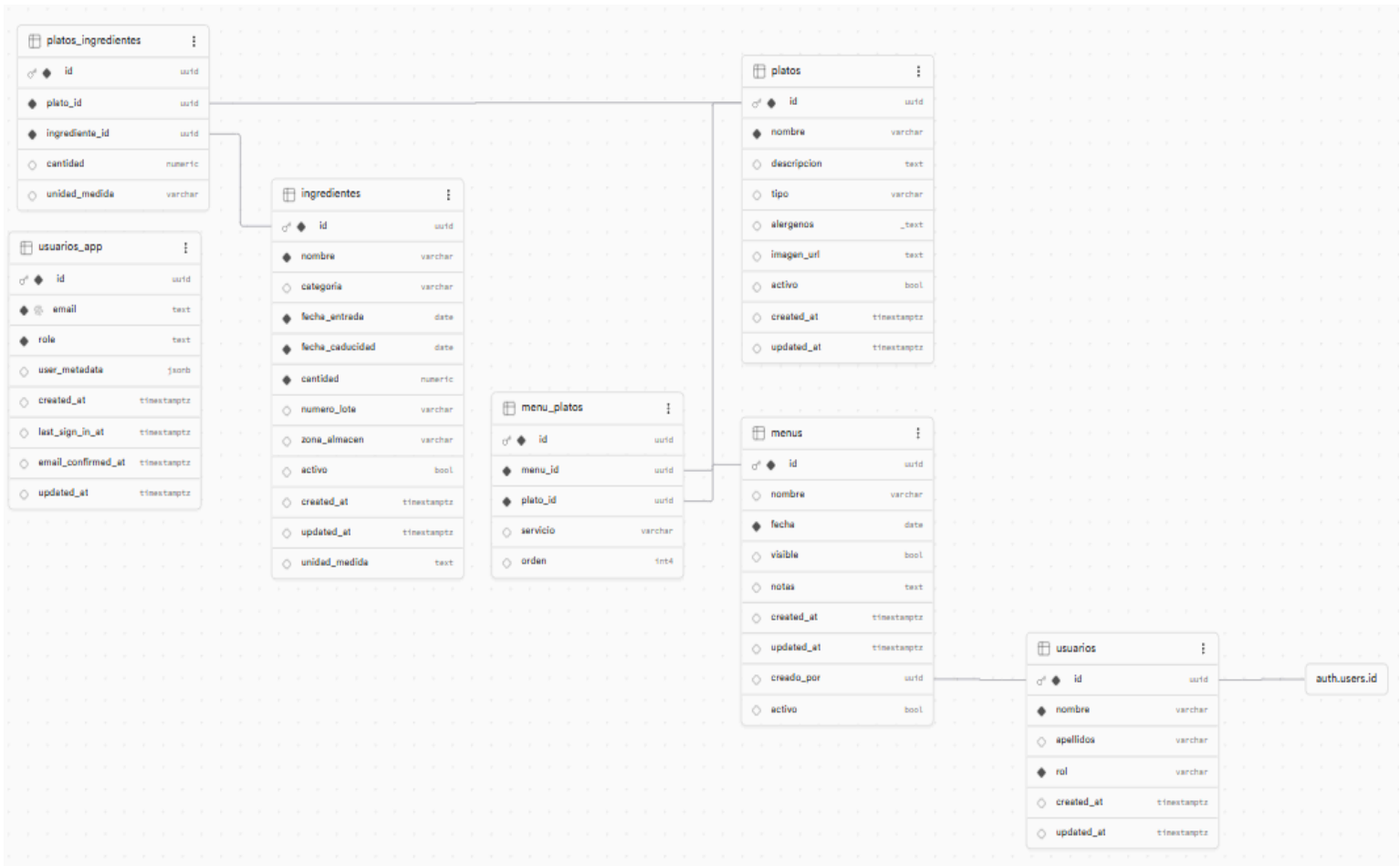
	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

- **src/assets/** Recursos multimedia importados por el código: imagen hero de la landing (image_hero.avif) y favicon (favicon.png).
- **src/components/** Componentes reutilizables de la interfaz. Se divide en componentes de la landing pública (Header, Hero, Features, Roles, Footer), el componente de acceso (Login), la cabecera de la aplicación autenticada (AppHeader) y los paneles especializados para la construcción de platos y menús (PanelIngredientes, PanelPlato, PanelPlatosSeleccionables, PanelMenuBuilder).
- **src/hooks/** Hooks personalizados de React. Contiene useRole.ts, que consulta la tabla usuarios_app en Supabase para obtener el rol real del usuario autenticado.
- **src/lib/** Inicialización de clientes externos. supabaseClient.ts crea el cliente público con la clave anónima y supabaseAdmin.ts crea el cliente privilegiado con la service_role key para operaciones de administración de usuarios.
- **src/pages/** Páginas completas de la aplicación, una por ruta. Incluye FormularioPage (registro de ingredientes), ListadoPage (inventario), GenerarPlatosPage (creación y edición de platos), PlatosPage (listado de platos), GenerarMenusPage (diseño de menús), GestionMenusPage (administración de menús), PublicMenuPage (vista pública del menú del día), AdminUsersPage (gestión de usuarios) y ForgotPasswordPage (recuperación de contraseña).
- **src/services/** Capa de acceso a datos. Abstrae todas las operaciones con Supabase: platoService.ts (CRUD de platos, validación de stock, normalización de unidades), menuService.ts (CRUD de menús, activación del menú diario) y userService.ts (creación, actualización de roles, eliminación y gestión de contraseñas de usuarios).
- **src/styles/** Hojas de estilo globales. index.css importa Tailwind CSS y define las animaciones globales. colors.css define las clases de color para los badges de categorías de ingredientes y zonas de almacén, además de los estilos de paginación y buscador.
- **src/types/** Definiciones de tipos TypeScript compartidas entre capas. plato.ts (interfaces de ingredientes y platos), menu.ts (interfaces de menús y relaciones plato-menú) y usuario.ts (interfaces de usuarios y formularios).



4. Estructura de Datos y Modelo Entidad-Relación

El sistema reposa en una base de datos relacional **PostgreSQL** con **Supabase**. Aunque la arquitectura es ágil, a grandes rasgos requiere:



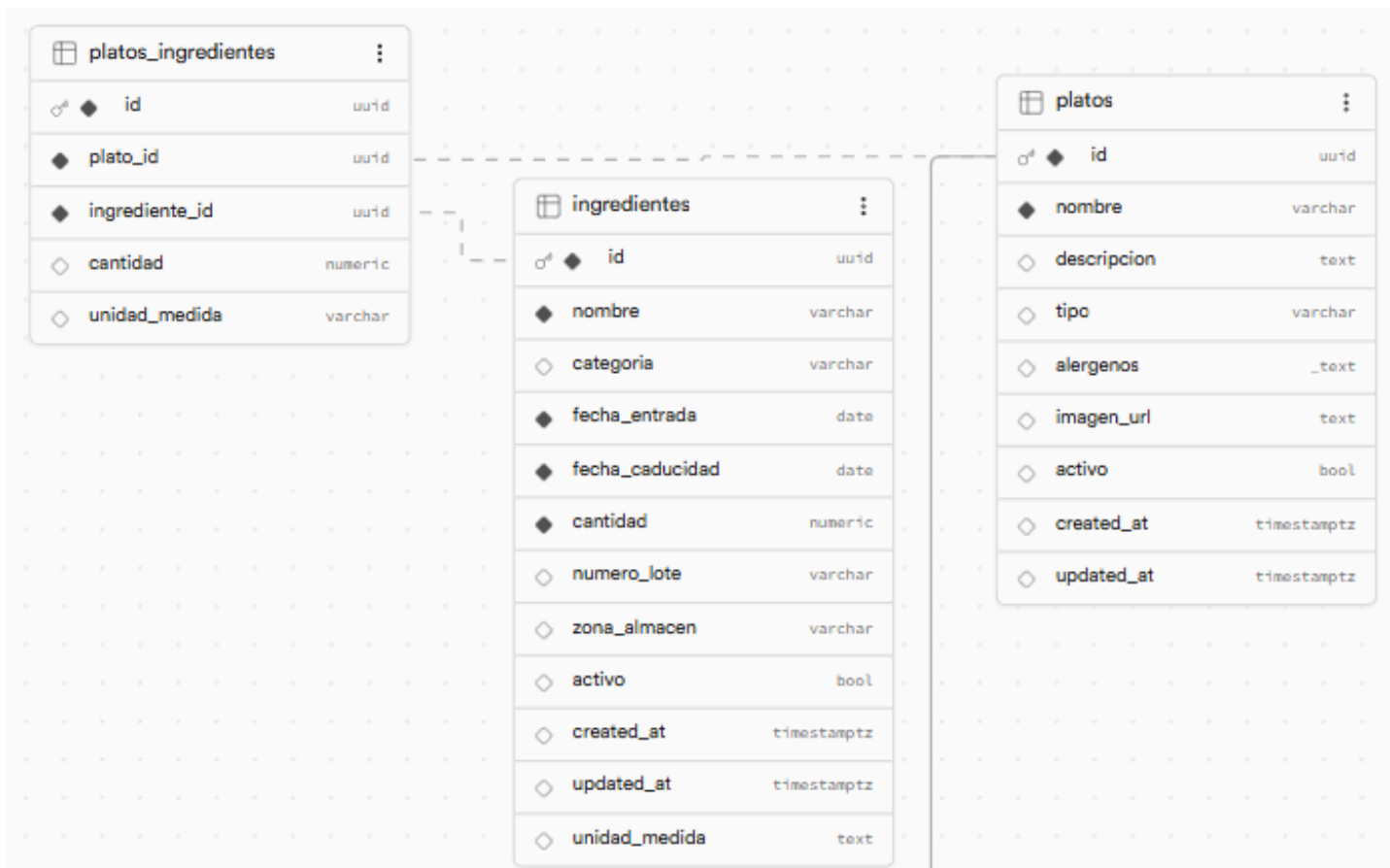
	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

- **auth.users (tabla nativa de Supabase):** La gestiona Supabase automáticamente. Almacena las credenciales de acceso de cada usuario (email, hash de contraseña, tokens de sesión). El campo id es el identificador único que se utiliza como referencia en el resto del sistema. No se accede directamente desde el código de la aplicación.
- **usuarios_app:** Es la tabla de usuarios propia de la aplicación. Su id coincide exactamente con el id de auth.users, sincronizándose en el momento de crear un usuario. Almacena el role del usuario (admin, jefecocina, cocinero, admin_usuarios), que es la única fuente de verdad para el control de acceso. El campo user_metadata es un JSON libre que replica información auxiliar del rol.
- **usuarios:** Tabla secundaria que recoge el perfil nominal del usuario (nombre, apellidos y un campo rol en formato varchar). Está presente en el diagrama de Supabase pero el código de la aplicación no la utiliza directamente para el control de acceso; ese rol lo gestiona usuarios_app.

usuarios_app		
NAME	COMMAND	APPLIED TO
allow_all_delete	DELETE	public
allow_all_insert	INSERT	public
allow_all_select	SELECT	public
allow_all_update	UPDATE	public

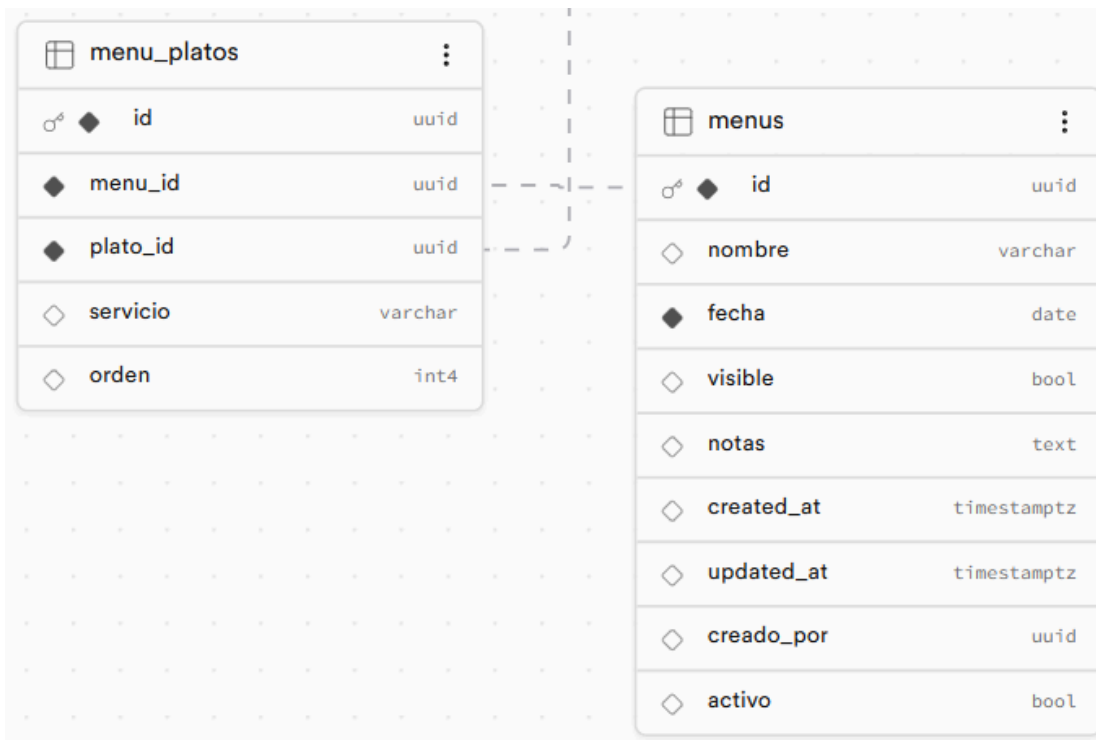
	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

- **ingredientes:** Tabla central del inventario. Registra cada alimento con todos los campos necesarios para la trazabilidad: nombre, categoría, fechas de entrada y caducidad, cantidad disponible, unidad de medida, número de lote y zona de almacén. El campo activo implementa el borrado lógico (los registros nunca se eliminan físicamente).
- **platos:** Almacena las recetas creadas por el equipo de cocina. Incluye nombre, descripción, URL de la imagen (almacenada en Supabase Storage), tipo y un campo alergenicos previsto para información alérgena. También usa borrado lógico mediante activo.
- **platos_ingredientes:** Tabla de unión entre platos e ingredientes. Representa la relación muchos-a-muchos: un plato tiene varios ingredientes y un ingrediente puede aparecer en varios platos. Para cada combinación guarda la cantidad requerida y la unidad de medida específica para esa receta.



	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

- **menus:** Almacena los menús diarios creados. Cada menú tiene un nombre, una fecha, notas opcionales y dos flags: activo (indica si es el menú del día que se muestra al público) y visible. El campo creado_por es una clave foránea hacia auth.users.id que registra qué usuario creó el menú.
- **menu_platos:** Tabla de unión entre menus y platos. Permite que un menú contenga varios platos y que un mismo plato aparezca en distintos menús. Para cada relación guarda el servicio (Primero, Segundo, Postre, etc.) y el orden de presentación dentro del menú.



Relación	Tipo	Descripción
auth.users→usuarios_app	1 a 1	Cada usuario de Supabase tiene exactamente un perfil en la app
auth.users → menus	1 a N	Un usuario puede crear múltiples menús

	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

platos→ platos_ingredientes	1 a N	Un plato tiene uno o más ingredientes
ingredientes→ platos_ingredientes	1 a N	Un ingrediente puede usarse en varios platos
menus → menu_platos	1 a N	Un menú contiene uno o más platos
platos → menu_platos	1 a N	Un plato puede aparecer en varios menús

5. Requisitos Funcionales y No Funcionales

Requisitos Funcionales

RF-01 — Autenticación y acceso El sistema permite a los usuarios iniciar sesión mediante correo electrónico y contraseña. Incluye recuperación de contraseña por email. Los usuarios no autenticados son redirigidos automáticamente a la página de inicio.

RF-02 — Gestión de ingredientes Los usuarios con rol autorizado pueden registrar nuevos ingredientes con los campos: nombre, categoría, fecha de entrada, fecha de caducidad, cantidad, unidad de medida, número de lote y zona de almacén. También pueden editar y eliminar (borrado lógico mediante el campo activo) cualquier ingrediente existente.

RF-03 — Listado e inventario El sistema muestra todos los ingredientes activos en una tabla paginada (10 elementos por página) con filtrado por nombre en tiempo real. Cada fila permite edición inline y eliminación con confirmación modal.

RF-04 — Creación de platos Los usuarios pueden crear platos seleccionando ingredientes del inventario, especificando cantidad y unidad de medida por ingrediente,

	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

añadiendo nombre, descripción e imagen. El sistema valida que no se supere el stock disponible antes de guardar.

RF-05 — Gestión de platos El sistema lista todos los platos activos y permite consultarlos, editarlos (redirige a la página de edición) y eliminarlos (borrado lógico). Al visualizar un plato se muestran sus ingredientes y cantidades.

RF-06 — Creación de menús Los usuarios autorizados pueden diseñar menús diarios seleccionando platos previamente creados, asignando a cada uno un servicio (Primero, Segundo, Postre, Entrante, Bebida u Otro), definiendo nombre, fecha y notas del menú.

RF-07 — Gestión de menús El sistema lista todos los menús creados. Permite activar un menú como "Menú Diario" (desactivando automáticamente el anterior), desactivar el menú activo, editar cualquier menú existente y eliminar menús con confirmación modal.

RF-08 — Menú público Existe una ruta pública (/menu-diario) accesible sin autenticación que muestra el menú activo del día, agrupado por servicios, con nombre, descripción e imagen de cada plato. Si no hay menú activo, muestra un mensaje informativo.

RF-09 — Control de acceso por roles El sistema implementa cuatro roles con permisos diferenciados:

RF-10 — Gestión de usuarios El rol **administrador** puede crear nuevos usuarios (email, contraseña y rol), cambiar el rol de cualquier usuario, cambiar la contraseña de un usuario directamente, enviar email de restablecimiento de contraseña y eliminar usuarios del sistema.

	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

Requisitos No Funcionales

RNF-01 — Tecnología frontend La aplicación está construida con React 19, TypeScript 5.9 y Vite 7 como bundler. El sistema de estilos utiliza Tailwind CSS 4 combinado con CSS personalizado.

RNF-02 — Backend y base de datos Se utiliza Supabase como plataforma Backend-as-a-Service, que provee base de datos PostgreSQL, autenticación, Row Level Security (RLS), almacenamiento de imágenes y cliente en tiempo real.

RNF-03 — Seguridad Los roles se validan en el servidor mediante la tabla usuarios_app en Supabase, no a partir de metadatos del cliente. Las rutas protegidas comprueban autenticación y rol antes de renderizar. Las operaciones administrativas sobre usuarios utilizan un cliente con service_role key que nunca se expone públicamente. La clave de servicio solo se usa en operaciones internas de administración.

RNF-04 — Rendimiento La paginación del listado limita las consultas a 10 registros por página. Las imágenes de platos se almacenan en Supabase Storage y se sirven mediante URL pública. El filtrado de ingredientes y platos se realiza en el cliente con useMemo para evitar renders innecesarios.

RNF-05 — Usabilidad La interfaz es completamente responsiva. Las acciones destructivas (eliminar ingrediente, plato o menú) requieren confirmación explícita mediante modal. Los mensajes de éxito y error se muestran con retroalimentación visual inmediata y desaparecen automáticamente.

RNF-06 — Accesibilidad pública La página /menu-diario es accesible sin autenticación y está diseñada para ser utilizada directamente por clientes del restaurante, opcionalmente mediante un código QR.

	CHEFMANAGER	JORGE ALEGRE MAESTRE - 2º DAW B
	MANUAL TÉCNICO	

RNF-07 — Mantenibilidad El código está organizado en capas separadas (componentes, páginas, servicios, tipos y hooks). La lógica de acceso a Supabase se abstrae completamente en los módulos de servicios, desacoplando de los componentes visuales.

RNF-08 — Validación de stock Antes de crear o actualizar un plato, el sistema normaliza las unidades de medida (g→kg, ml→L, etc.) y compara la cantidad requerida con el stock disponible, rechazando la operación si no hay suficiente inventario.